



# UD04: Análisis de sistemas robotizados

Modelos de Inteligencia Artificial

version: 2026-08-24

# ¿Qué veremos?

1. Métodos y aplicaciones de la robótica
2. Qué clase de problema resuelve
3. Modelado y control cinemático
4. Problemas y soluciones
5. Percepción robótica
6. Incertidumbre y aprendizaje
7. Técnicas de programación
8. Humanos y robots
9. Diseño e implementación

# RA4 y sus criterios de evaluación

“ **RA4** — Analiza sistemas robotizados, evaluando opciones de diseño e implementación. ”

| CE       | Criterio                                                                   | Dónde |
|----------|----------------------------------------------------------------------------|-------|
| <b>a</b> | Recopilar los problemas del modelado y control cinemático en manipuladores | §1-3  |
| <b>b</b> | Buscar soluciones a los problemas de los robots                            | §4-6  |
| <b>c</b> | Valorar las técnicas de programación de robots y de sistemas robotizados   | §7-8  |
| <b>d</b> | Evaluar opciones de diseño e implementación                                | §9    |

Cuatro contenidos oficiales para cuatro CE: en este RA **encajan casi uno a uno**, y es la excepción del módulo.

## Al terminar la unidad serás capaz de...

- **Clasificar** robots por tipo y describir aplicaciones reales con datos.
- **Distinguir** los sensores por lo que miden y elegir el adecuado.
- **Resolver** la cinemática directa con parámetros DH y entender por qué la inversa es más difícil.
- **Identificar** singularidades, redundancia y límites, y sus soluciones.
- **Plantear** un problema de planificación en el espacio de configuración.
- **Explicar** cómo un robot se localiza y hace un mapa con sensores ruidosos.
- **Comparar** técnicas de programación resolviendo el **mismo** problema de cuatro formas.
- **Aplicar** criterios de diseño: payload, alcance, sensores, seguridad.

# 1. Métodos y aplicaciones de la robótica

RA4-a

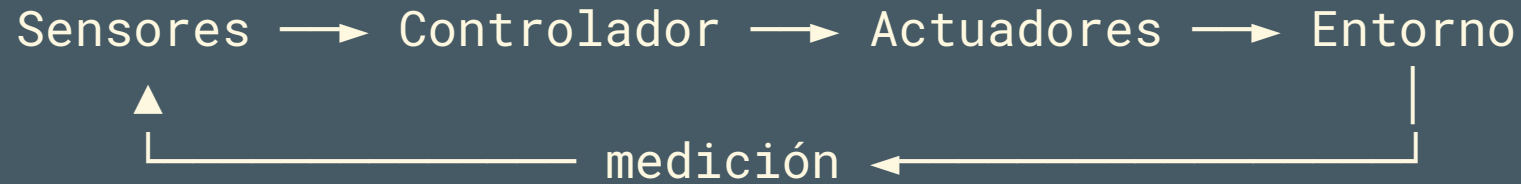
# Un robot es un agente encarnado

Es el único sistema de IA del curso que **cambia el estado del mundo físico**.

- Un clasificador que falla produce una etiqueta mala.
- Un robot que falla **rompe una pieza**. O algo peor.

Y su entorno es **parcialmente observable, estocástico y con más agentes dentro**: las cámaras no ven tras las esquinas, los engranajes patinan y las personas son impredecibles.

# Percibir, procesar, actuar



Los **efectores** ejercen fuerza y pueden cambiar tres cosas:

- el estado del **robot**: el coche gira las ruedas y avanza,
- el del **entorno**: el brazo empuja una taza,
- el de las **personas**: alguien se aparta al verlo llegar.

# Tipos de robot

| Tipo                                 | Qué es                                                                         |
|--------------------------------------|--------------------------------------------------------------------------------|
| <b>Manipulador</b>                   | Un brazo; no necesita cuerpo, se atornilla a una mesa o al suelo               |
| <b>Móvil con ruedas</b>              | Aspiradora, AGV de almacén, coche autónomo, rover                              |
| <b>Con patas</b>                     | Terreno accidentado donde la rueda no llega                                    |
| <b>Aéreo (UAV) y submarino (AUV)</b> | Cuadricópteros, exploración oceánica                                           |
| <b>Cobot</b>                         | Comparte espacio con personas: potencia y fuerza limitadas                     |
| <b>Otros</b>                         | Prótesis, exoesqueletos, alas, <b>enjambres</b> , <b>entornos inteligentes</b> |

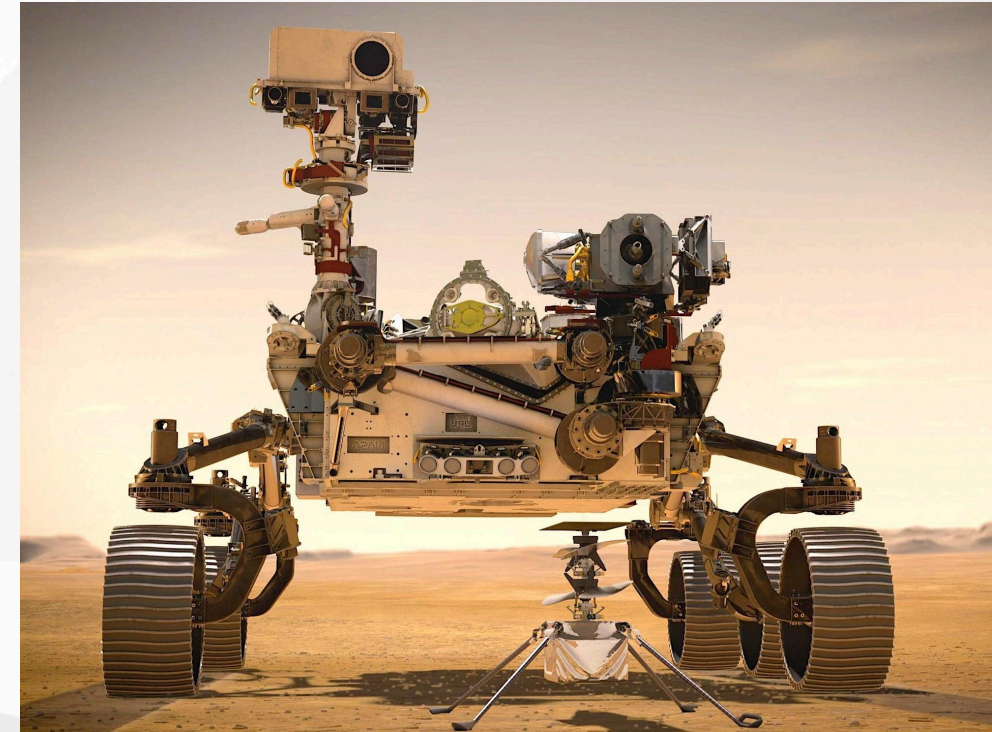
El robot antropomórfico de la ficción es el **menos** frecuente en la realidad.

# No es solo el tamaño

Un manipulador de **una tonelada** y un brazo montado en una silla de ruedas se diferencian en más que la escala:

- el primero mueve **mucha carga**,
- el segundo mueve poca pero es **seguro entre personas**.

**Payload y seguridad son criterios distintos**, y a menudo opuestos.



# Sensores: dos clasificaciones a la vez

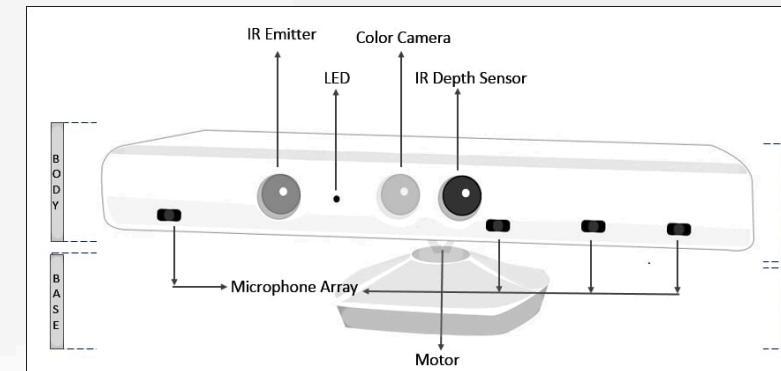
## Por cómo obtienen la señal:

- **Pasivos** — cámaras: captan lo que el entorno emite.
- **Activos** — sonar, lidar: emiten energía y miden lo que vuelve. Dan más información, pero consumen más y **se interfieren entre sí**.

**Por lo que miden:** el entorno • la ubicación del robot • su configuración interna.

# Qué informa cada clase

| Clase                       | Sensores                                                                                          |
|-----------------------------|---------------------------------------------------------------------------------------------------|
| Del entorno                 | Sonar, visión estéreo, luz estructurada (Kinect), tiempo de vuelo, <b>lidar</b> , radar, táctiles |
| De la ubicación             | GPS/GLONASS, balizas de interior, señal wifi, balizas de sonar                                    |
| De la configuración interna | Encoders, odometría de rueda, giroscopios, fuerza y par                                           |



## Las cifras que deciden

| Sensor                    | Alcance y precisión                                                                                                            |
|---------------------------|--------------------------------------------------------------------------------------------------------------------------------|
| Lidar de barrido          | ~1 cm a 100 m. El de referencia en coche autónomo                                                                              |
| Cámara de tiempo de vuelo | Rango a <b>60 fps</b> ; peor que el lidar a plena luz                                                                          |
| Radar                     | Hasta <b>kilómetros</b> , y <b>ve a través de la niebla</b>                                                                    |
| GPS                       | <b>Unos metros</b> ; con GPS diferencial, milimétrico en condiciones ideales. <b>No funciona en interiores ni bajo el agua</b> |
| Fuerza y par              | 3 traslaciones + 3 rotaciones, <b>cientos de medidas por segundo</b>                                                           |

# Por qué la odometría no basta

Contar vueltas de rueda parece barato y exacto. Y lo es... **durante unos metros.**

- Las ruedas **derrapan y patinan.**
- El error **se acumula sin límite:** nada lo corrige.

Por eso se combina siempre con giroscopios y con **una referencia externa.** Es la razón de ser de la localización probabilística del bloque 5.

# El caso de la bombilla

Un manipulador de **una tonelada** tiene que enroscar una bombilla. Es facilísimo romperla.

- Los sensores de **fuerza** dicen con qué fuerza agarra.
- Los de **par**, con qué fuerza gira.
- Midiendo **cientos de veces por segundo**, corrige **antes** de romper el cristal.

Manipular con cuidado no viene de tener un brazo más suave: viene de **medir rápido**.

# Actuadores

| Tipo              | Dónde se usa                           |
|-------------------|----------------------------------------|
| <b>Eléctrico</b>  | El más común: articulaciones de brazos |
| <b>Hidráulico</b> | Mucha fuerza: maquinaria pesada        |
| <b>Neumático</b>  | Movimientos rápidos y simples, pinzas  |

Mueven articulaciones de **revolución** (giran, ángulo  $\theta$ ) o **prismáticas** (deslizan, distancia  $d$ ). Las dos, de un eje.

# Pinzas: el compromiso

- **Mordaza paralela** — dos dedos, un actuador.  
«Amada y odiada por su simplicidad».
- **Tres dedos** — algo más de flexibilidad sin complicarse.
- **Mano antropomórfica** — la Shadow Hand tiene **20 actuadores**: puede girar el móvil en la mano, y es mucho más difícil de controlar.



# Más grados de libertad no es mejor

Cada actuador multiplica lo que el robot **puede** hacer **y** lo que hay que **decidir** en cada instante.

Veinte actuadores es un espacio de acción enorme.

Casi toda la robótica industrial funciona con **dos dedos y un actuador**, porque la tarea no necesita más.

## La robótica, con datos (IFR 2025)

| Dato                                 | Valor (2024)                                            |
|--------------------------------------|---------------------------------------------------------|
| Robots industriales instalados       | <b>542.000</b> (4.º año sobre 500.000)                  |
| Stock operativo mundial              | <b>4,66 millones</b> (+9 %)                             |
| Líder en densidad                    | <b>Corea del Sur</b> (1.012-1.220 por 10.000 empleados) |
| Líder de mercado                     | <b>China</b> (295.000, el 54 %)                         |
| <b>España</b> , 3.er mercado europeo | <b>5.100 unidades</b> , por la automoción               |
| Robótica médica                      | <b>+91 %</b>                                            |

## Cuatro casos reales

| Robot                  | Características                             | Uso                                             |
|------------------------|---------------------------------------------|-------------------------------------------------|
| <b>KUKA KR AGILUS</b>  | 6-11 kg, 726-1.101 mm                       | 60 sándwiches/min; 3.000 muestras de sangre/día |
| <b>FANUC</b>           | Hasta <b>2,3 t</b> , alcance 4,7 m          | Paletizado (M-410, 700 kg), soldadura           |
| <b>UR e-Series</b>     | 3-16 kg; repetibilidad <b>±0,03-0,05 mm</b> | Montaje asistido, inspección                    |
| <b>Amazon Robotics</b> | Más de <b>1 millón</b> de robots            | Logística de almacén                            |

# Pensar en «robot + tarea»

No se elige el robot más caro. Se define **primero la tarea**:

- qué pieza, qué peso,
- qué ritmo, qué precisión,
- qué entorno.

Y **después** se busca el modelo que cumple payload, alcance, repetibilidad y seguridad.

Es el criterio del bloque 9 y del Taller 2.

## **2. Qué clase de problema resuelve la robótica**

# Todas las dificultades a la vez

La robótica es **no determinista**, **parcialmente observable** y **multiagente**.

Y lo multiagente no es competir **o** cooperar: es **las dos cosas**. En un pasillo donde solo cabe uno, el robot y la persona **colaboran** — ninguno quiere chocar— y **compiten** por pasar primero.

Un robot demasiado educado **se queda atascado para siempre**.

# ¿De quién es la recompensa?

Si el robot lleva la comida a un paciente, la recompensa es **del paciente**, no suya.

Y ahí está el problema: **la recompensa verdadera está en la cabeza del usuario**. El robot tiene que deducirla, o conformarse con la aproximación que le programó un ingeniero.

Es el problema de alineación de la UD06, con un brazo de una tonelada.

# La jerarquía de tres niveles

En crudo, un robot ve píxeles y envía corrientes a motores. Entre eso y «lleva la comida a la 12» hay un abismo. Se salva **partiendo el problema**:

```
Planificación de tareas → submetas discretas: ir a la puerta, abrirla
    ↓
Planificación de movimiento → un camino sin colisiones
    ↓
Control → que los actuadores sigan ese camino
```

# Lo que se pierde al partirlo

Dividir reduce la complejidad, pero **renuncia a que las partes se ayuden:**

- La **acción** puede mejorar la percepción: moverse para ver mejor.
- El movimiento **óptimo sobre el papel** puede ser el peor de seguir.
- Un plan de tareas puede ser **imposible de instanciar** en movimiento.

Por eso hoy se avanza en cada nivel **y en volver a integrarlos.**

# **3. Modelado y control cinemático**

**RA4-a**

# La cadena cinemática

| Concepto                | Definición                                                           |
|-------------------------|----------------------------------------------------------------------|
| Grado de libertad (DoF) | Movimiento independiente. Hacen falta $\geq 6$ para pose libre en 3D |
| Revolución (R)          | Gira: su variable es un ángulo $\theta$                              |
| Prismática (P)          | Desliza: su variable es una distancia $d$                            |
| Espacio articular       | El vector <b>q</b> con la posición de cada articulación              |
| Espacio cartesiano      | La <b>pose</b> : posición más orientación del efector                |

# Cuatro configuraciones

| Configuración        | Articulaciones | Uso típico                           |
|----------------------|----------------|--------------------------------------|
| Articulado           | $\geq 3R$      | El más común en industria (6 ejes)   |
| Cartesiano / pórtico | 3P             | Manipulación simple, gran alcance    |
| SCARA                | RRP            | Montaje: rígido en Z, flexible en XY |
| Delta / paralelo     | Paralelas      | Empaquetado de alta velocidad        |



# Cinemática directa

De los ángulos a la pose: **pose = f(q)**. Se multiplican **matrices homogéneas 4×4**.

$${}^0T_n = {}^0T_1 \cdot {}^1T_2 \cdot \dots \cdot {}^{n-1}T_n$$

Cada transformación se describe con los **parámetros de Denavit-Hartenberg**: cuatro valores por articulación –  $\theta, d, a, \alpha$ .

Se escribe la tabla DH del robot y la pose sale de encadenar las matrices.

## Tabla DH del Puma 560 (fragmento)

| Articulación | $\theta$ | $d$    | $a$    | $\alpha$    |
|--------------|----------|--------|--------|-------------|
| 1            | $q_1$    | 0,6718 | 0      | $-90^\circ$ |
| 2            | $q_2$    | 0      | 0,4318 | $0^\circ$   |
| 3            | $q_3$    | 0,1500 | 0,0203 | $90^\circ$  |
| 4            | $q_4$    | 0,4318 | 0      | $-90^\circ$ |

`fkine([0, 0.2, 0.3, 0.4, 0.5, 0.6])` devuelve una matriz 4×4 con la pose: del orden de `[0,25; -0,13; 1,15]`.

## Un brazo plano 3R, a mano

$$x = l_1 \cos \theta_1 + l_2 \cos(\theta_1 + \theta_2) + l_3 \cos(\theta_1 + \theta_2 + \theta_3)$$

$$y = l_1 \sin \theta_1 + l_2 \sin(\theta_1 + \theta_2) + l_3 \sin(\theta_1 + \theta_2 + \theta_3)$$

Con  $l_1 = l_2 = l_3 = 1$  y los tres ángulos a 0  $\rightarrow$  el efector está en **(3, 0)**.

Poniendo  $\theta_1 = 90^\circ \rightarrow$  pasa a **(0, 3)**.

La cinemática directa **siempre tiene una única solución**.

# La inversa es otra historia

$q = f^{-1}(\text{pose})$ . Aquí están casi todos los problemas que pide recopilar el CE a:

| Problema                    | En qué consiste                                            |
|-----------------------------|------------------------------------------------------------|
| <b>Múltiples soluciones</b> | Un 6R general: <b>hasta 16</b> . Con muñeca esférica, 8    |
| <b>Redundancia</b>          | Más DoF de los necesarios → <b>infinitas</b> soluciones    |
| <b>Sin solución</b>         | El objetivo está fuera del alcance                         |
| <b>Singularidad</b>         | El jacobiano pierde rango → velocidad articular a infinito |

Métodos **analíticos** (Pieper, desacoplamiento) o **numéricos** (Newton-Raphson, pseudoinversa, CCD, FABRIK).

# Controlar el movimiento

| Estrategia                                | Cómo                                                         |
|-------------------------------------------|--------------------------------------------------------------|
| <b>Posición</b>                           | PID por eje: medida del encoder, consigna el ángulo objetivo |
| <b>Velocidad</b> ( <i>resolved-rate</i> ) | $\dot{q} = J^+ v$ , con la pseudoinversa del jacobiano       |
| <b>Fuerza / impedancia</b>                | El robot se comporta como masa-muelle-amortiguador           |

Servomotores **con encoder**, en lazo cerrado: sin realimentación no hay forma de saber si el eje llegó.

# De P a par calculado

- **P** — acción proporcional al error. Simple, pero **oscila**.
- **PD** — añade la derivada del error: **amortigua** la oscilación.
- **PID** — añade la integral: corrige el **error sistemático persistente**.
- **Par calculado** — usa la **dinámica inversa** para calcular el par necesario y deja al PID solo lo que el modelo no acierta.

Lo último es lo que usan los robots industriales de verdad.

# De dónde salen las trayectorias

No se salta de un punto a otro: **se interpola**.

- **jtraj** – en el espacio articular, polinomio de 5.º grado: aceleración y *jerk* continuos, movimiento suave.
- **ctrjaj** – **línea recta cartesiana**, que es lo que hace falta cuando el efector lleva una herramienta de corte.

El controlador va siguiendo esa trayectoria punto a punto.

# 4. Problemas y soluciones

RA4-b

# Las cuatro singularidades

| Singularidad | Cuándo ocurre                                    |
|--------------|--------------------------------------------------|
| De muñeca    | Los ejes 4 y 6 se alinean ( $\theta_5 = 0$ )     |
| De hombro    | El centro de la muñeca corta el eje de la base   |
| De codo      | El brazo, del todo estirado o plegado            |
| De límite    | El efector llega al borde del volumen de trabajo |

# Qué se ve de verdad en la célula

No es un problema abstracto de álgebra. Las velocidades articulares **tienden a infinito**, así que el servo pide una corriente que no puede dar:

- **sobrecorriente**,
- **vibraciones**,
- o una **parada de seguridad** a mitad del ciclo.

Se evitan al planificar, o se cruzan bajando la velocidad.

# Las soluciones

| Problema             | Solución                                                                   |
|----------------------|----------------------------------------------------------------------------|
| Singularidades       | Planificar evitándolas; medir la <b>manipulabilidad</b> (Yoshikawa)        |
| IK compleja          | <b>Desacoplamiento</b> : la muñeca esférica separa posición de orientación |
| Múltiples soluciones | Criterios: evitar obstáculos, minimizar recorrido                          |
| Límites articulares  | Modelarlos y restringir el <i>solver</i>                                   |
| Precisión deficiente | <b>Calibrar</b> (hasta $\times 10$ de mejora) y medir con la ISO 9283      |

# Precisión no es repetibilidad

Un robot puede ser **repetible** y a la vez **impreciso**: vuelve siempre al mismo punto, pero **ese punto no es el que se programó**.

- Con *teach pendant* basta la repetibilidad: el punto se enseñó ahí.
- Con programación **offline**, las coordenadas vienen de un CAD y la precisión es crítica → **hay que calibrar**.

Repetibilidad de los cobots UR e-Series:  **$\pm 0,03-0,05$  mm.**

# El espacio de configuración

Cambio de punto de vista: en vez del robot moviéndose por una habitación, **un punto moviéndose por el espacio de todas sus configuraciones posibles.**

- Brazo de 6 ejes → **6 dimensiones.**
- Robot móvil en un plano → 3 (x, y, orientación).

Los obstáculos se vuelven **regiones prohibidas**; lo que queda es el **espacio libre**. El problema deja de ser geométrico: es **buscar un camino.**

# El problema de la mudanza del piano

Se llama así por el esfuerzo de una empresa de mudanzas para llevar un piano grande e irregular de una habitación a otra **sin golpear nada**.

Formalmente: un espacio  $W$ , obstáculos  $0 \subset W$ , un robot con configuración  $C$ , una configuración inicial  $q_s$  y una objetivo  $q_g$ . Se busca un **camino continuo** por el espacio libre.

Se complica de tres formas: objetivo como **conjunto**, **coste** a minimizar, o **restricciones** — si lleva una taza de café, mantenerla vertical todo el trayecto.

## Cuatro formas de planificar

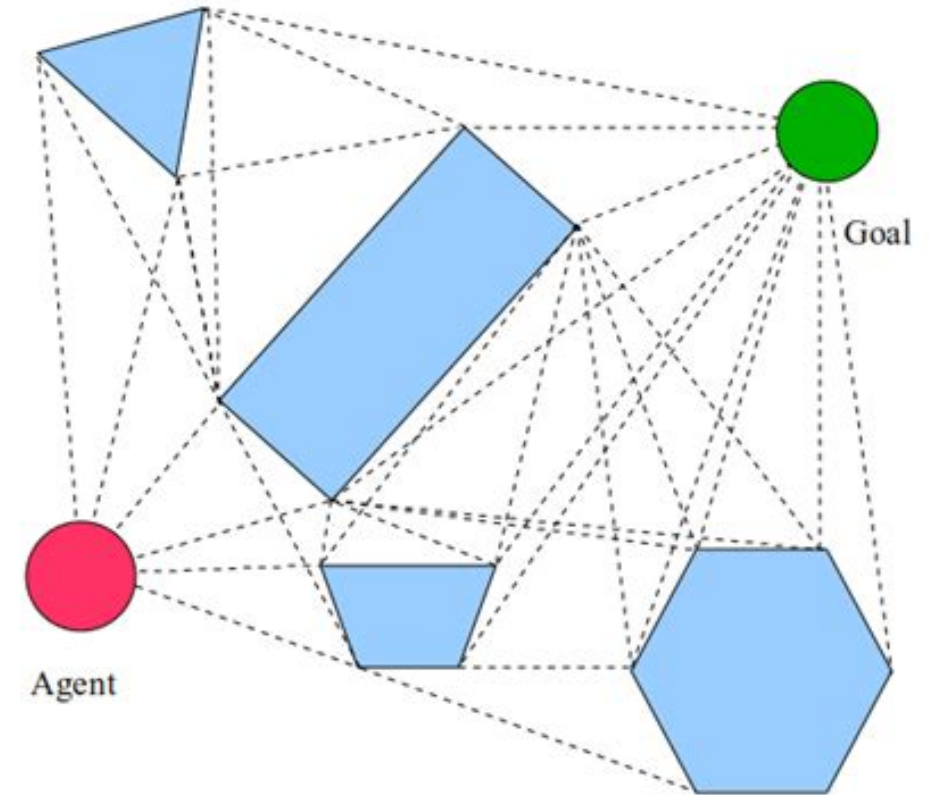
| Método                        | Idea                                                  | Fuerte                                          | Flojo                                            |
|-------------------------------|-------------------------------------------------------|-------------------------------------------------|--------------------------------------------------|
| <b>Grafo de visibilidad</b>   | Vértices de obstáculos + líneas de visión; después A* | El camino <b>más corto</b> en 2D                | Escala mal; <b>roza</b> las esquinas             |
| <b>Voronoi</b>                | Circular por los bordes entre regiones                | Maximiza la distancia: <b>seguro</b>            | Caminos más largos                               |
| <b>Descomposición celular</b> | Trocear el espacio libre y buscar en el grafo         | Simple y completo                               | Las celdas <b>explotan</b> al subir de dimensión |
| <b>Muestreo (RRT, PRM)</b>    | Lanzar configuraciones al azar y conectar             | Lo único práctico con <b>muchas dimensiones</b> | No es óptimo; hay que suavizar                   |

# Corto o seguro

El **grafo de visibilidad** busca el camino más corto: pasa **pegado a las esquinas**.

**Voronoi** busca el más seguro: **se aleja de todo**, y por eso es más largo.

Cuál conviene depende de si el riesgo es **chocar** o es **tardar**.



# Plan y política no son lo mismo

- Un **plan** dice **qué camino** recorrer.
- Una **política** dice **qué acción tomar desde cualquier estado** al que llegues.

Las leyes de control del bloque 3 son **políticas**, no planes.

Y la robótica real hace una **componenta en dos pasos**.

# La componenda, y su precio

1. **Planificar** en un espacio simplificado — solo cinemática, sin dinámica. Sale la trayectoria de referencia.
2. **Convertirla en política**: un controlador que la sigue y **vuelve a ella** al desviarse.

Dos suboptimalidades:

- planificar **sin la dinámica**: el camino más corto en el papel puede ser el peor de seguir;
- suponer que **si te desvías lo mejor es volver al plan**, cuando a veces conviene replanificar.

El **control óptimo** (LQR, iLQR) ataca las dos a la vez.

# 5. Percepción robótica

RA4-b

## Tres problemas, de menos a más

| Problema            | Se sabe             | Se busca                |
|---------------------|---------------------|-------------------------|
| <b>Localización</b> | El mapa             | Dónde está el robot     |
| <b>Mapeo</b>        | Dónde está el robot | El mapa                 |
| <b>SLAM</b>         | Nada de las dos     | <b>Las dos a la vez</b> |

SLAM es el caso realista, y parece imposible: para saber dónde estás necesitas el mapa, y para el mapa necesitas saber dónde estás.

# Se rompe con probabilidad

En vez de **una respuesta**, el robot mantiene una **distribución de probabilidad** sobre dónde puede estar —su *estado de creencia*— y la afina con cada medida.

Las herramientas son las de toda la IA con incertidumbre:

- **filtros de Kalman,**
- **modelos ocultos de Markov,**
- **redes bayesianas.**

# Monte Carlo, en tres fotos

Una nube de «partículas», cada una una hipótesis:

1. Al arrancar, **repartidas por todo el plano**.
2. Llegan medidas y se **agrupan**: quedan **varios grupos**, porque los pasillos se parecen.
3. Con suficientes medidas, **colapsan en un solo sitio**.

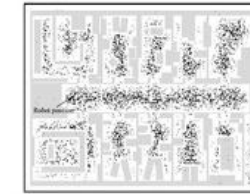


Fig. 2: Global localization. Initialization.

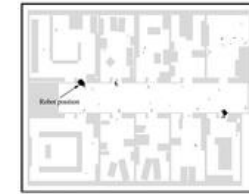


Fig. 3: Ambiguity due to symmetry.

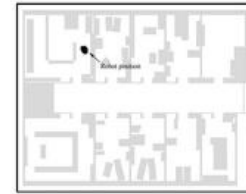


Fig. 4: Successful localization.

# Lo interesante es el paso 2

El robot representa **explícitamente que hay varias respuestas posibles**, en vez de escoger una y equivocarse.

Y solo necesita dos modelos: el del **movimiento** y el del **sensor**.

# Pero no siempre hace falta la artillería

La tendencia va hacia representaciones con **semántica** —no «aquí hay algo», sino «esto es una puerta»— y las técnicas probabilísticas **ganan** en los problemas difíciles.

Aun así, a veces son **demasiado engorrosas** y una solución simple es igual de efectiva.

**EX2** y **EX3** navegan sobre los píxeles de una cámara **sin ningún filtro probabilístico**, y funcionan.

# Percepción que se readapta

Entras de la calle soleada a una habitación con fluorescentes.

No solo hay menos luz: el fluorescente tiene **más componente verde**, así que **todos los colores cambian**.

Y no nos parece que a la gente se le haya puesto la cara verde: nuestra percepción **se readapta en segundos**.

Un robot que no haga eso **creerá que ha cambiado de mundo**.

# 6. Incertidumbre y aprendizaje

RA4-b

# El atajo que usan los robots reales

La mayoría usa **algoritmos deterministas**, con dos atajos:

1. **Discretizar** el espacio continuo (visibilidad, celdas).
2. Ante la duda, **quedarse con el estado más probable**.

Es más rápido y escala mejor. El precio: **el robot actúa como si estuviera seguro cuando no lo está**.

# Cuándo se rompe el atajo

Funciona mientras la distribución tenga **un solo pico claro**.

Falla justo en el paso 2 de Monte Carlo: cuando hay **varias hipótesis igual de buenas** —tres pasillos que se parecen—, quedarse con la más probable es **tirar una moneda...**

...y el robot actúa con plena confianza sobre una respuesta que tiene un **33 %** de ser cierta.

# Por qué el refuerzo falla en un robot real

| En simulación |                              | En el robot real                                                                                      |
|---------------|------------------------------|-------------------------------------------------------------------------------------------------------|
| Velocidad     | Millones de pruebas en horas | Las mismas tardarían <b>años</b>                                                                      |
| Riesgo        | Ninguno                      | <b>No puede arriesgarse</b> a la prueba que lo dañaría — y por tanto <b>no puede aprender de ella</b> |

**El mundo real se niega a ir más rápido que el tiempo real.**

# De ahí el problema *sim-to-real*

Transferir a un robot real lo aprendido en simulación es un **área de investigación activa**.

Y por eso los sistemas prácticos incorporan **conocimiento previo** del robot, del entorno y de la tarea: es la única forma de aprender rápido **y** comportarse con seguridad mientras aprende.

**EX4**, **EX5** y **EX6** hacen este recorrido en pequeño **y en simulación**, que es donde se puede.

# 7. Técnicas de programación de robots RA4-c

## Las cinco formas de programar un robot

| Técnica                               | Ventaja                             | Desventaja                                                 | Cuándo                            |
|---------------------------------------|-------------------------------------|------------------------------------------------------------|-----------------------------------|
| <b>Teach pendant</b>                  | Curva mínima, puesta a punto rápida | <b>Para la producción</b> mientras programas               | Paletizado, soldadura por puntos  |
| <b>Guiado manual</b>                  | Intuitivo, sin código               | Solo en cobots seguros                                     | Montaje asistido                  |
| <b>Textual</b> (RAPID, KRL, URScript) | Determinista, integra con PLC       | Propietario, <b>no portable</b>                            | Células estables de alta cadencia |
| <b>Offline (OLP)</b>                  | <b>Cero impacto en producción</b>   | Licencias; exige modelos precisos y <b>robot calibrado</b> | Geometrías complejas              |
| <b>ROS 2</b>                          | Abierto, acceso a algoritmos de IA  | Curva alta                                                 | Investigación, multi-robot        |

# URScript se parece a Python

Los robots de **Universal Robots** se programan en **URScript**, un lenguaje muy parecido a Python.

Y se puede enviar al robot desde un cliente Python por **FTP, SSH o RTDE**.

Es la puerta de entrada natural a la programación de cobots desde este módulo.



# «Colaborativo» no es del hardware

La **ISO 10218:2025** prohíbe llamar «robot colaborativo» a un brazo aislado.

Lo que puede ser colaborativa es la **aplicación completa**: robot + herramienta + entorno + tarea.

**Un cobot con un cuchillo en la pinza no es una aplicación colaborativa.**

Es un matiz legal, y es el que decide si hace falta vallado.

# Comparar de verdad: el mismo problema, cuatro veces

Un robot con cámara tiene que **seguir una línea** en el suelo. Y el problema se resuelve cuatro veces.

EX2 reglas → EX3 difusa → EX4 datos → EX5 red neuronal → EX6 NEAT

Eso es lo que pide el CE c: **valorar características diferenciadoras.**

## Qué cambia en cada salto

| Entregable | Técnica       | Qué escribes tú                 | Qué sale del programa                               |
|------------|---------------|---------------------------------|-----------------------------------------------------|
| EX2        | Reglas        | Todas las condiciones, a mano   | Nada: el comportamiento es el que programaste       |
| EX3        | Lógica difusa | Variables lingüísticas y reglas | La transición <b>suave</b> entre ellas              |
| EX5        | Red neuronal  | Arquitectura y entrenamiento    | El comportamiento, aprendido de <b>tus</b> ejemplos |
| EX6        | NEAT          | Solo la función de aptitud      | La red <b>y su topología</b> , sin ejemplos         |

# Lo que hay que observar

No es cuál «funciona mejor». Es **qué se gana y qué se pierde**:

- cuánto código escribes,
- cuánto tienes que entender del problema,
- cuántos datos necesitas,
- y, lo más importante, **si puedes explicar por qué el robot hizo lo que hizo**.

Las reglas de **EX2** se leen. Los pesos de la red de **EX5**, no.

Es la tensión entre interpretabilidad y potencia de la UD05 – y en la UD06 se vuelve un problema legal.

# 8. Humanos y robots

RA4-c • RA4-d

# Una persona no es un obstáculo móvil

Cuando el robot comparte espacio con personas aparece un problema que no es de mecánica ni de control.

La forma habitual de plantearlo es como un **juego**: eso supone explícitamente que las personas son **agentes con objetivos**.

Pero suponerlo **no** significa que sean **perfectamente racionales**.

# El «¿qué crees que creo que crees?»

El juego es difícil para el robot... **y para nosotros.**

Obliga a pensar en lo que hará el robot en respuesta a lo que hace la persona, que depende de lo que el robot cree que hará la persona, que depende de...

Las personas **no gestionamos eso**, así que nos comportamos de forma **subóptima** de maneras bastante predecibles.

**Un robot que asuma racionalidad perfecta predecirá mal.**

# La solución práctica

Se descompone en dos:

1. **Predicción** — usar lo que la persona está haciendo ahora para estimar qué hará después.
2. **Acción** — usar esa predicción para calcular el movimiento del robot.

# Aprender lo que la persona quiere

El usuario **no sabría escribir** la función objetivo perfecta. El robot tiene que **inferirla observando**: qué corrige, qué acepta, qué repite.

Y esto no es un detalle técnico: un robot que optimiza una **aproximación mala** del objetivo hace **exactamente lo que se le pidió** y no lo que se quería.

Con un brazo de una tonelada, **especificar mal el objetivo es un modo de fallo**, igual que una singularidad.

# 9. Diseño e implementación

RA4-d

## Los cinco criterios de selección

| Criterio             | Pregunta guía                             | Dato típico                    |
|----------------------|-------------------------------------------|--------------------------------|
| <b>Payload</b>       | ¿Qué masa mueve, contando la herramienta? | UR3e 3 kg ... FANUC 2,3 t      |
| <b>Alcance</b>       | ¿Qué distancia máxima?                    | KUKA 726-1.101 mm; FANUC 4,7 m |
| <b>Repetibilidad</b> | ¿Cuánta dispersión al volver?             | UR e-Series $\pm 0,03-0,05$ mm |
| <b>Precisión</b>     | ¿Coincide con lo programado?              | Mejora al calibrar             |
| <b>Entorno</b>       | ¿Polvo, humedad, atmósfera explosiva?     | Versiones IP / ATEX            |

# El payload no es el peso de la pieza

Es la pieza **más** la herramienta, la brida, los cables y los sensores acoplados.

Y además hay que comprobar los **momentos de inercia** de la herramienta contra los límites de las reductoras de la muñeca.

Un robot puede aguantar **10 kg pegados a la brida** y no aguantar **6 kg** en el extremo de una herramienta larga.

## Ejemplo guiado: pieza de 1,5 kg a 700 mm

1. **Payload** — pieza 1,5 + pinza 0,5 + cables  $\approx$  **2,2 kg**. Hace falta  $\geq 2,5$  kg y  $\geq 700$  mm: un **UR5e** (5 kg / 850 mm) cumple.
2. **Repetibilidad** — la tarea tolera  $\pm 0,1$  mm; el UR5e da  $\pm 0,05$ . Y **no hace falta calibrar**, porque los puntos se enseñan.
3. **Singularidades** — se simula y se comprueba que la trayectoria no pasa por codo estirado ni muñeca alineada.
4. **Seguridad** — ¿comparte espacio? → aplicación colaborativa. ¿No? → vallado con enclavamientos.

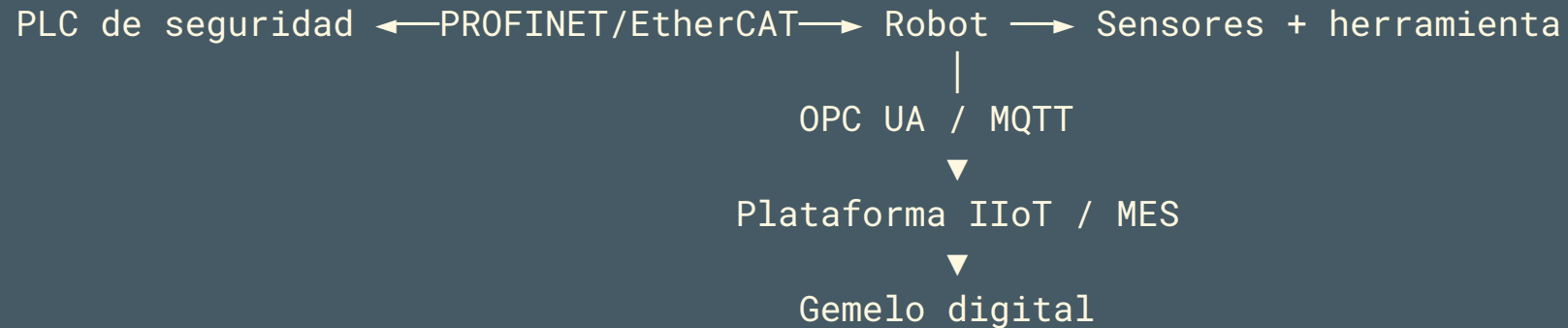
# Si hay que cruzar una singularidad...

...se cambia el layout, no el controlador.

Es más barato mover el transportador que pelearse con el robot.

Y el orden importa: **primero la tarea, después el robot**. Nunca al revés.

# La célula y la Industria 4.0



El bus del PLC es **determinista**; la telemetría va por OPC UA o MQTT. Con esos datos, el gemelo digital anticipa fallos —fatiga de reductoras— **antes** de que paren la línea.

# Seguridad y normativa

| Norma                 | Qué regula                                                                 |
|-----------------------|----------------------------------------------------------------------------|
| <b>ISO 12100</b>      | Evaluación de riesgos de máquinas: la base                                 |
| <b>ISO 10218-1/-2</b> | Robots industriales: requisitos de seguridad                               |
| <b>ISO/TS 15066</b>   | Cobots: límites biomecánicos                                               |
| <b>ISO 10218:2025</b> | <b>Vigente:</b> absorbe la 15066 y obliga a evaluar la aplicación completa |
| <b>ISO 9283</b>       | Cómo se mide repetibilidad y precisión                                     |

# Lo que cambió en 2025

La **ISO 10218:2025** hace dos cosas:

1. Absorbe los límites biomecánicos de la ISO/TS 15066 y los vuelve **obligatorios**.
2. Añade **auditorías de ciberseguridad industrial** de la célula.

Un robot conectado a la red de planta es una **superficie de ataque**: es el *security by design* de la UD06, aplicado a una máquina que se mueve.

# Dos simuladores, dos trabajos

**roboticstoolbox-python** — para la cinemática de manipuladores: más de 50 robots reales con su cinemática resuelta.

```
robot = rtb.models.Panda()  
pose = robot.fkine([0, -0.8, 0.8, 0, 0.8, 0, 0])  
q = robot.ikine_LM(pose)
```

**aitk.robots** — para el robot móvil con cámara, dentro del propio notebook.

```
world = bots.World(220, 180, ground_image_filename="pista.png")  
robot = bots.Scribbler(x=100, y=90, a=90)  
robot.add_device(bots.Camera(64, 32))
```

# Cierre

## Puntos clave (I)

- Un robot es un **agente encarnado**, en un entorno parcialmente observable, estocástico y multiagente.
- Los sensores se clasifican por **cómo** miden y por **qué** miden. La **odometría se degrada sin límite**: hay que corregirla.
- **Más grados de libertad no es mejor**: 20 actuadores pueden más y son mucho más difíciles de controlar.
- La robótica parte el problema en **tarea** → **movimiento** → **control**, y con ello renuncia a que los niveles se ayuden.
- La **cinemática directa** tiene solución única; la **inversa**, hasta **16**, más redundancia y singularidades.
- **Precisión no es repetibilidad**: se puede volver siempre al mismo punto equivocado.

## Puntos clave (II)

- Planificar es buscar camino en el **espacio de configuración**: visibilidad da el **más corto**, Voronoi el **más seguro**, el muestreo es el único viable en muchas dimensiones.
- Un **plan** dice qué camino; una **política**, qué hacer desde cualquier estado. La robótica real planifica y luego convierte el plan en política, pagando dos suboptimalidades.
- **SLAM** mantiene una **distribución de probabilidad**, no una respuesta.
- El **refuerzo** falla en el robot real: el mundo no va más rápido que el tiempo real y no puede arriesgar la prueba que lo dañaría.
- **«Colaborativo» es de la aplicación**, no del hardware (ISO 10218:2025).
- Diseñar es **emparejar la tarea con el modelo** y verificarlo en simulación **antes** de comprar.

# Las cuatro sesiones

| Semana | Contenido                                                                      | CE      |
|--------|--------------------------------------------------------------------------------|---------|
| 15     | Métodos y aplicaciones; hardware; qué problema resuelve la robótica            | a       |
| 16     | Cinemática directa e inversa, singularidades; Taller 1                         | a,<br>b |
| 17     | Espacio de configuración y planificación; percepción y SLAM; OpenCV, EX1 - EX3 | b, c    |
| 18     | Técnicas comparadas ( EX4 - EX6 ); célula, seguridad y normativa; Taller 2     | c, d    |

## Cómo se evalúa

| Peso | Instrumento                                                                          |
|------|--------------------------------------------------------------------------------------|
| 40 % | Media de los <b>seis entregables</b> ( EX1 - EX6 ), cada uno con su rúbrica sobre 10 |
| 60 % | Prueba del RA4: test y desarrollo sobre el contenido de la unidad                    |

Los entregables son una **secuencia**: EX4 genera los datos de EX5. No conviene dejarlos para el final ni saltarse el orden.

La **normativa exige alcanzar todos los RA**; el centro lo concreta en  **$\geq 5$  en cada uno**.

## ¿Y ahora?

Un robot percibe, razona y **actúa sobre el mundo físico**. Es donde la IA deja de equivocarse en una etiqueta y empieza a equivocarse en una pieza.

**A seguir una línea con una cámara.**